Haxudio Lead Management System - PRD

Nov 6, 2026 - Prepared for: Syed Ali

1. Overview

Haxudio generated 13,000 business leads via the Hermes agent and stored them as JSON documents in MongoDB. There is currently no interface to work this data: the team cannot read, claim, filter, or update leads, and there is no way to see who is already approaching a given lead.

This PRD defines an internal, login-only web application for Haxudio's team (under 50 users) to browse, claim, filter, and update these leads, with full accountability on who touched which lead and when.

Problem: 13k leads sitting in raw JSON with no usable interface; no visibility into which teammate is already working a lead, causing duplicate outreach risk.

Solution: A lightweight internal CRM-style tool (Notion-inspired UI) backed by the existing MongoDB collection, with claim/status tracking, activity logging, filtering, and an admin monitoring panel.

Out of scope for this app: generating new leads, editing the Hermes scraping pipeline, public-facing pages, or a landing page (none needed — users land directly on login or dashboard).

2. Users and Roles

Two roles only. No signup flow — admin creates/seeds user accounts.

Role	Can do
Regular User	Log in, view lead list, view lead details (contact info hidden until claimed), claim a lead, change lead status, add notes, filter/sort/search leads, see own dashboard stats
Admin	Everything a Regular User can do, plus: view list of all users, view each user's profile (leads viewed, leads claimed, leads completed/closed), view org-wide leaderboard. Admin does not bulk-edit or delete users from this version — monitoring only, as specified.

No public signup. No landing page. Visiting the app with no valid session redirects straight to `/login`; a valid session redirects straight to `/dashboard`.

3. Lead Data Model

Existing fields (from the Hermes/MongoDB collection) — the app reads/writes against this schema, it does not change it:

`business_name`, `category`, `suburb`, `city`, `state`, `full_address`, `country`, `postcode`, `google_rating`, `phone_numbers[]`, `emails[]`, `website{}`, `socials{instagram, facebook, linkedin, tiktok, youtube}`, `agency_analysis{needs_services[], pitch_reason, lead_quality_score}`, `score_v2{total, bucket, contact, gap, social, value, why[], completeness}`, `source{}`, `notes`, `created_at`, `updated_at`.

New fields this app adds to each lead document:

Field	Type	Purpose
`status`	enum string	New / Viewed / Claimed / Contacted / Follow-up / Converted / Deal Closed / Not Interested / Rejected
`claimed_by`	user ref (nullable)	Which user currently owns this lead
`claimed_at`	datetime (nullable)	When it was claimed — used for stale auto-release
`viewed_by[]`	array of {user, viewed_at}	Everyone who has opened this lead, for the "someone already looked at this" signal
`notes`	reused/extended	Free-text notes; each note entry stores author + timestamp
`activity_log[]`	array of {type, user, timestamp, detail}	Full audit trail: viewed / claimed / status_changed / note_added / unclaimed_stale
`is_archived`	boolean, default false	Soft delete flag — archived leads are hidden from the default list but never physically deleted

A separate `users` collection holds: `name`, `email`, `password_hash`, `role` (admin/user), `created_at`, plus aggregate counters (`leads_viewed_count`, `leads_claimed_count`, `leads_completed_count`) maintained by the backend as actions happen.

4. Lead Status Workflow

Status	Set by	Trigger
New	System	Default on all 13k imported leads
Viewed	System (automatic)	Any user opens the lead detail page
Claimed	System (automatic)	User clicks "Reveal Contact Info" on that lead
Contacted	User (manual)	User manually updates status after reaching out
Follow-up	User (manual)	Needs a second touchpoint
Converted	User (manual)	Lead agreed to engage Haxudio's services
Deal Closed	User (manual)	Deal signed/closed
Not Interested	User (manual)	Lead declined
Rejected	User (manual)	Disqualified/bad lead

Progress grouping (used for stats and dashboard):

- **New** — untouched
- **In Progress** — Viewed, Claimed, Contacted, Follow-up
- **Closed / Completed** — Converted, Deal Closed, Not Interested, Rejected

A lead can only move to Claimed by one user at a time (`claimed_by` is single-owner). If another user opens an already-claimed lead, they see a banner: "Claimed by {name} — {time ago}" and the contact-reveal button is disabled for them unless they are Admin or the claim has gone stale (see 5.7).

5. Core Features (MVP)

5.1 Lead List + Pagination

- 20 leads per page.
- Pagination controls: **Back** button, **Next** button, and a page-number input box in the middle ("Go to page __") that jumps directly on Enter.
- List view shows: business_name, category, city/state, google_rating, status badge, score_v2.bucket, claimed_by (if any).

5.2 Search

- Free-text search across: `business_name`, `email`, `phone_numbers`, `city`.

5.3 Filters and Sort

- Filter by: `status`, `category`, `city` / `suburb` / `state`, `score_v2.bucket` (A/B/C), `google_rating` (range), `website.design_condition` (dated/modern), social presence flags (has Instagram / has Facebook / website dead vs live), `claimed_by` (mine / unclaimed / specific user, admin only).
- Sort by: `score_v2.total` (desc default), `google_rating`, `created_at`, `updated_at`.

5.4 Lead Detail View

- Shows full lead record, score_v2 breakdown, socials, website tech info.
- **Contact info (phone_numbers, emails) is hidden by default** behind a "Reveal Contact Info" button.
- Opening the detail page alone sets status → Viewed and logs a `viewed` activity entry (does not reveal contact info).
- Clicking "Reveal Contact Info" sets status → Claimed, sets `claimed_by` / `claimed_at`, and logs a `claimed` activity entry. Once claimed by someone else, other regular users see contact fields blurred/locked with the claim banner.

5.5 Status Updates + Notes

- Only the user who claimed the lead (or Admin) can change its status past Claimed, or add notes.
- Notes are free text, timestamped, author-tagged, and appended (not overwritten) — shown as a running note history on the lead.
- Every status change and note addition writes an `activity_log` entry.

5.6 Lead Activity Timeline

- Each lead detail page shows a chronological log, e.g.:
  - "Viewed by Ali Jaan — Nov 6, 5:40 PM"
  - "Claimed by Ali Jaan — Nov 6, 5:42 PM"
  - "Status changed to Contacted by Ali Jaan — Nov 6, 6:10 PM"
  - "Note added by Ali Jaan: 'Called, no response' — Nov 6, 6:11 PM"

5.7 Stale Claim Auto-Release

- If a claimed lead has no status change or note added for 72 hours, a scheduled job auto-reverts status to Viewed, clears `claimed_by`, and logs "Auto-unclaimed (stale, 72h no activity)" — making it available to the rest of the team again.

5.8 Dashboard Summary Cards

- Each user's `/dashboard` shows: My claimed leads (active), Leads I viewed today, Leads I completed (closed statuses), and a quick link into "My claimed leads" list.

5.9 Soft Delete / Archive

- No hard deletes anywhere. An "Archive" action sets `is_archived: true` on a lead; archived leads are excluded from default list/filter results but remain in the database and are recoverable by Admin.

6. Admin Panel

Admin-only section (`/admin`), monitoring only — no bulk edit/delete.

- **User list:** name, email, role, date joined, last login.
- **Per-user profile:** total leads viewed, total leads claimed, total leads completed (closed statuses), currently active claims, note count.
- **Org-wide leaderboard:** users ranked by leads completed (optional toggle: by leads claimed).
- **Global stats:** total leads in system, leads by status breakdown, leads archived count.

7. Authentication and Security

- **Login:** email + password only. No OAuth/social login.
- **Password rules:** minimum 8 characters, at least 1 uppercase, 1 lowercase, 1 number. Hashed with **bcrypt** (cost factor 10-12).
- **Tokens: JWT** access token (short-lived, ~15 min) + refresh token (long-lived, ~7 days, stored httpOnly secure cookie). Access token renewed silently via refresh token; refresh token revoked on logout/password change.
- **Forgot password:** email-based reset link (time-limited token), resets password.
- **Change password:** available from account settings while logged in (requires current password).
- **Session redirect rule:** valid session → hitting `/` or `/login` redirects straight to `/dashboard` . No valid session → any protected route redirects straight to `/login` . No landing page exists.
- **Rate limiting:** 30 requests per 15 minutes per IP (applied at minimum to auth endpoints: login, register, forgot-password, reset-password).

- **Authorization:** every API route checks JWT + role; lead-mutating routes additionally check `claimed_by` ownership (or Admin override) before allowing status/notes changes.

8. Non-Functional Requirements

- **Performance:** no SEO needed (login-gated app). Priority is perceived speed: list/filter queries indexed in MongoDB, paginated server-side (never fetch all 13k at once), response times should feel instant (<300ms typical for list/filter endpoints).
- **PWA:** fully online-only. Installable to home screen (manifest + icons), but **no service-worker caching and no offline support** — if there's no network, the app should simply fail to load, by design.
- **Language:** English only, no i18n.
- **UI/UX:** light mode only, no dark mode. Notion-inspired information density and interaction patterns (inline editing feel, clean typography, generous whitespace) — explicitly not a generic AI-template look (see Section 9 for direction).
- **Scale:** designed for ⩽50 concurrent internal users and ~13k-50k leads; no need to design for public-scale traffic.

9. Tech Stack and Design Direction

Stack

- Frontend: React (Vite), deployed on **Vercel** as a PWA (manifest.json + icons, no service worker caching).
- Backend: Node.js + Express.js, deployed on **Render**.
- Database: MongoDB (existing 13k-lead collection + new `users` collection).
- Auth: JWT (access + refresh), bcrypt password hashing.
- State/data fetching: React Query (or SWR) for server state; Context or Zustand only for light UI state (not Redux — overkill for this scope).

Design direction (critical — must not look AI-generic)

- Reference: **Notion** — calm, content-first, light mode, generous whitespace, clear typographic hierarchy, understated borders instead of heavy shadows/cards-on-cards.
- Palette: a single desaturated accent (e.g. deep slate-teal `#2B4C4A` or similar muted ink tone) on a warm off-white background (`#FAF9F6` -ish), with neutral grays for structure — **no purple-to-blue gradients, no glassmorphism, no generic SaaS-template blue**.

- Typography: one clean grotesk/humanist sans (e.g. Inter or a system font stack) with real weight contrast (heading weight 600-700, body 400) instead of default browser styling.
- Status badges, claim banners, and the activity timeline should feel like considered, custom components — not default shadcn colors left unstyled.
- No stock illustrations, no generic dashboard hero images, no decorative icons without purpose.

10. API Design (high-level)

Auth

- `POST /api/auth/login`
- `POST /api/auth/refresh`
- `POST /api/auth/logout`
- `POST /api/auth/forgot-password`
- `POST /api/auth/reset-password`
- `PATCH /api/auth/change-password`

Leads

- `GET /api/leads` — query params: `page`, `limit=20`, `search`, `status`, `category`, `city`, `bucket`, `minRating`, `claimedBy`, `sortBy`, `sortOrder`
- `GET /api/leads/:id` — fetch detail, auto-marks Viewed + logs activity
- `POST /api/leads/:id/claim` — reveals contact info, sets Claimed
- `PATCH /api/leads/:id/status` — update status (owner or admin only)
- `POST /api/leads/:id/notes` — append a note
- `GET /api/leads/:id/activity` — full activity timeline
- `PATCH /api/leads/:id/archive` — soft delete

Admin

- `GET /api/admin/users`
- `GET /api/admin/users/:id/stats`
- `GET /api/admin/leaderboard`
- `GET /api/admin/stats` — global counts

Dashboard

- `GET /api/dashboard/me` — current user's summary cards

11. Out of Scope (this version)

- Bulk actions (bulk status change, bulk archive)
- CSV export
- Duplicate lead detection
- Multi-language support
- Offline mode / service-worker caching
- Public landing page / marketing site
- Social login / OAuth
- Admin bulk user management (create/delete users is a manual/seed-level operation for now)

12. Environment Variables (`.env.example`)

```
# Server
PORT=5000
NODE_ENV=production
CLIENT_URL=https://<your-vercel-app>.vercel.app

# Database
MONGO_URI=mongodb+srv://<user>:<pass>@<cluster>/<dbname>

# Auth
JWT_ACCESS_SECRET=<random-64-char-string>
JWT_REFRESH_SECRET=<different-random-64-char-string>
JWT_ACCESS_EXPIRES=15m
JWT_REFRESH_EXPIRES=7d
BCRYPT_SALT_ROUNDS=10

# Rate limiting
RATE_LIMIT_WINDOW_MINUTES=15
RATE_LIMIT_MAX_REQUESTS=30

# Password reset email
SMTP_HOST=<smtp-provider-host>
SMTP_PORT=587
SMTP_USER=<smtp-user>
SMTP_PASS=<smtp-pass>
RESET_TOKEN_EXPIRES=1h
```

Agent ko bata den ke actual secrets `.env` mein jayenge, `.env.example` sirf keys ke liye repo mein commit hoga.

13. Folder Structure

```
client/                     (React + Vite PWA)
  src/
    components/
      leads/                LeadCard, LeadTable, StatusBadge, ClaimBanner
      activity/             ActivityTimeline
      common/               Button, Input, Modal, Pagination
    pages/
      Login.jsx
      ForgotPassword.jsx
      Dashboard.jsx
      LeadsList.jsx
      LeadDetail.jsx
      Admin.jsx
    hooks/                  useAuth, useLeads, useDebounce
    services/               api.js (axios instance), authService,
leadService, adminService
    context/                AuthContext.jsx
    utils/                  formatDate, statusColors
    App.jsx
  public/
    manifest.json
    icons/

server/                     (Node + Express)
  src/
    models/                 Lead.js, User.js
    controllers/            authController, leadController, adminController
    routes/                 authRoutes, leadRoutes, adminRoutes,
dashboardRoutes
    middleware/              authMiddleware (verify JWT), roleMiddleware,
rateLimiter, errorHandler
    services/                staleClaimJob.js (cron for 5.7)
    utils/                   tokenUtils, bcryptUtils
    config/                  db.js
    app.js
    server.js
  .env.example
```

14. Mongoose Schema Reference

```
// User.js
const userSchema = new Schema({
  name: { type: String, required: true },
  email: { type: String, required: true, unique: true, lowercase: true },
  passwordHash: { type: String, required: true },
  role: { type: String, enum: ['admin', 'user'], default: 'user' },
  resetToken: String,
  resetTokenExpires: Date,
  leadsViewedCount: { type: Number, default: 0 },
  leadsClaimedCount: { type: Number, default: 0 },
  leadsCompletedCount: { type: Number, default: 0 }
}, { timestamps: true });

// Lead.js  -- extends the existing Hermes JSON shape, does not replace it
const activityEntrySchema = new Schema({
  type: { type: String, enum: ['viewed', 'claimed', 'status_changed',
'note_added', 'unclaimed_stale'] },
  user: { type: Schema.Types.ObjectId, ref: 'User' },
  timestamp: { type: Date, default: Date.now },
  detail: String
}, { _id: false });

const leadSchema = new Schema({
  // ...all existing Hermes fields (business_name, category, phone_numbers,
emails,
  // website, socials, agency_analysis, score_v2, source, etc.) stay as-is,
type: Mixed
  // or mapped field-by-field as needed --

  status: {
    type: String,
    enum: ['New', 'Viewed', 'Claimed', 'Contacted', 'Follow-up', 'Converted',
'Deal Closed', 'Not Interested', 'Rejected'],
    default: 'New'
  },
  claimedBy: { type: Schema.Types.ObjectId, ref: 'User', default: null },
  claimedAt: { type: Date, default: null },
  viewedBy: [{
    user: { type: Schema.Types.ObjectId, ref: 'User' },
    viewedAt: { type: Date, default: Date.now }
  }],
  notesLog: [{
    author: { type: Schema.Types.ObjectId, ref: 'User' },
    text: String,
```

```
    createdAt: { type: Date, default: Date.now }
  }],
  activityLog: [activityEntrySchema],
  isArchived: { type: Boolean, default: false }
}, { timestamps: true, strict: false });

// Indexes for performance (Section 8)
leadSchema.index({ status: 1 });
leadSchema.index({ 'score_v2.bucket': 1 });
leadSchema.index({ claimedBy: 1 });
leadSchema.index({ isArchived: 1 });
leadSchema.index({ business_name: 'text', city: 'text' });
```